SOFTWARE TESTING
PRACTICAL REPORT

PRACTICAL REVIEW — PR-13

Student Name	Parth Fegade
Enrollment Number	2504030100083
Programme	BCA — Semester 3
Practical	PR-13 — Selenium Environment Setup / WebDriver Test
Tools	Python, Selenium WebDriver, Google Chrome, PyCharm

1. Aim

To launch Google Chrome using Selenium WebDriver, open the Selenium website, print the page title and current URL, and close the browser session.

2. Objectives

- Verify that Python and Selenium are available in the PyCharm project environment.
- Start Chrome through Selenium WebDriver and maximize the browser window.
- Navigate to https://www.selenium.dev/ and read the page title and current URL.
- Confirm that the browser automation script runs without an execution error.
- Close the browser session cleanly after the test.

3. Software and Requirements

Component	Purpose
Python	Runs the automation script.
Selenium WebDriver	Controls the browser using Python.
Google Chrome	Browser launched and controlled by WebDriver.
PyCharm	IDE used to write and execute the Python script.
Selenium website	Target page used to verify browser navigation.

4. Procedure

1. Open the Python project in PyCharm.
2. Create a Python file named pr13.py or tutorial13.py.
3. Install Selenium in the selected interpreter using: pip install selenium.
4. Write the Selenium script shown in Section 5.
5. Run the file from PyCharm.
6. Observe Chrome opening https://www.selenium.dev/ and check the title and URL printed in the Run console.
7. Confirm that the script ends and the browser closes.

5. Program Code

```
from selenium import webdriver

driver = webdriver.Chrome()
driver.maximize_window()

driver.get("https://www.selenium.dev/")

print("Browser launched successfully")
print("Page Title:", driver.title)
print("Current URL:", driver.current_url)

driver.quit()
```

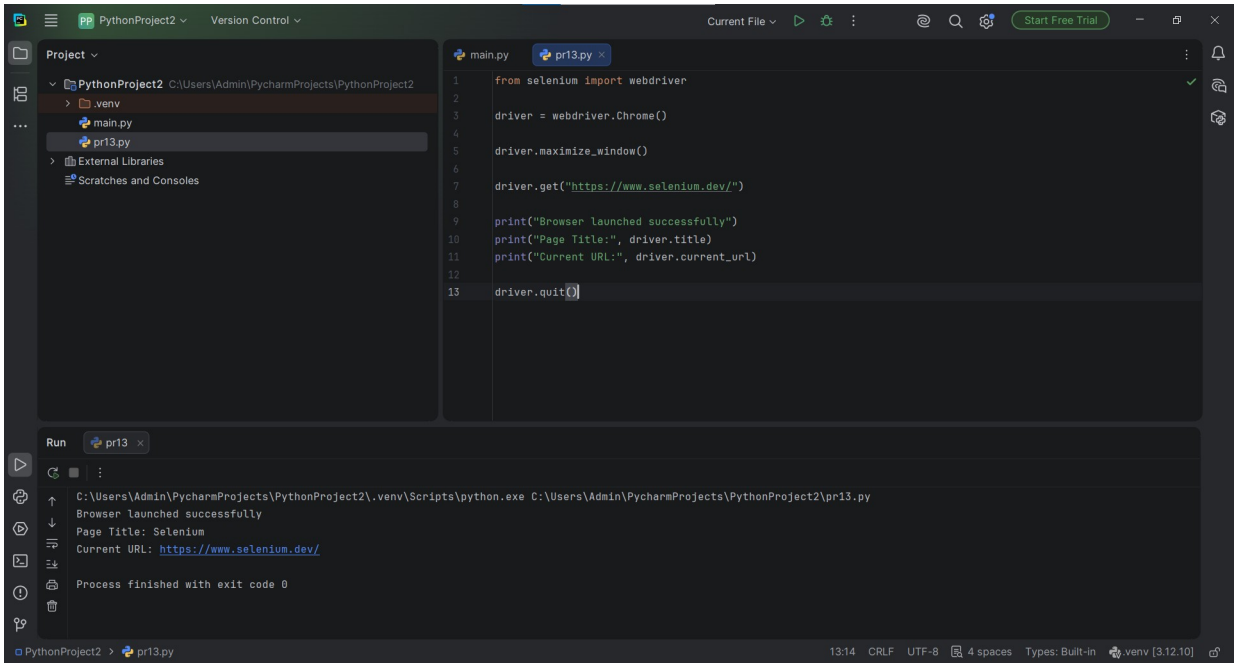
6. Code Explanation

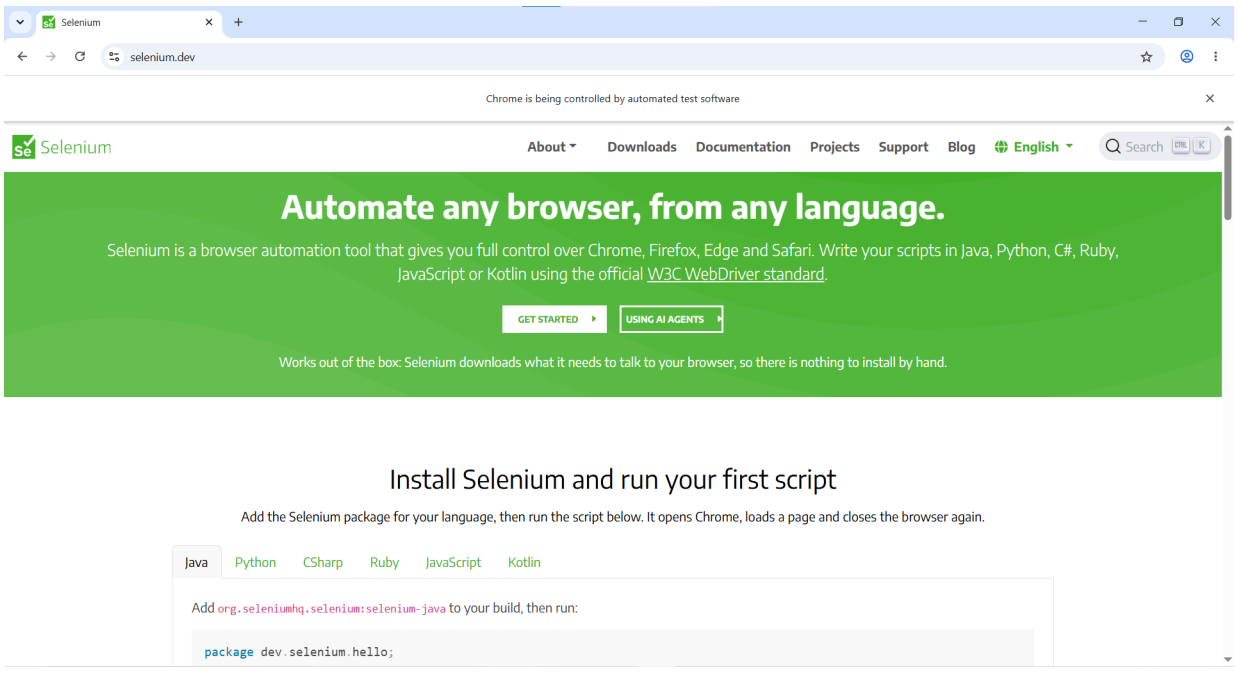
Code statement	Explanation

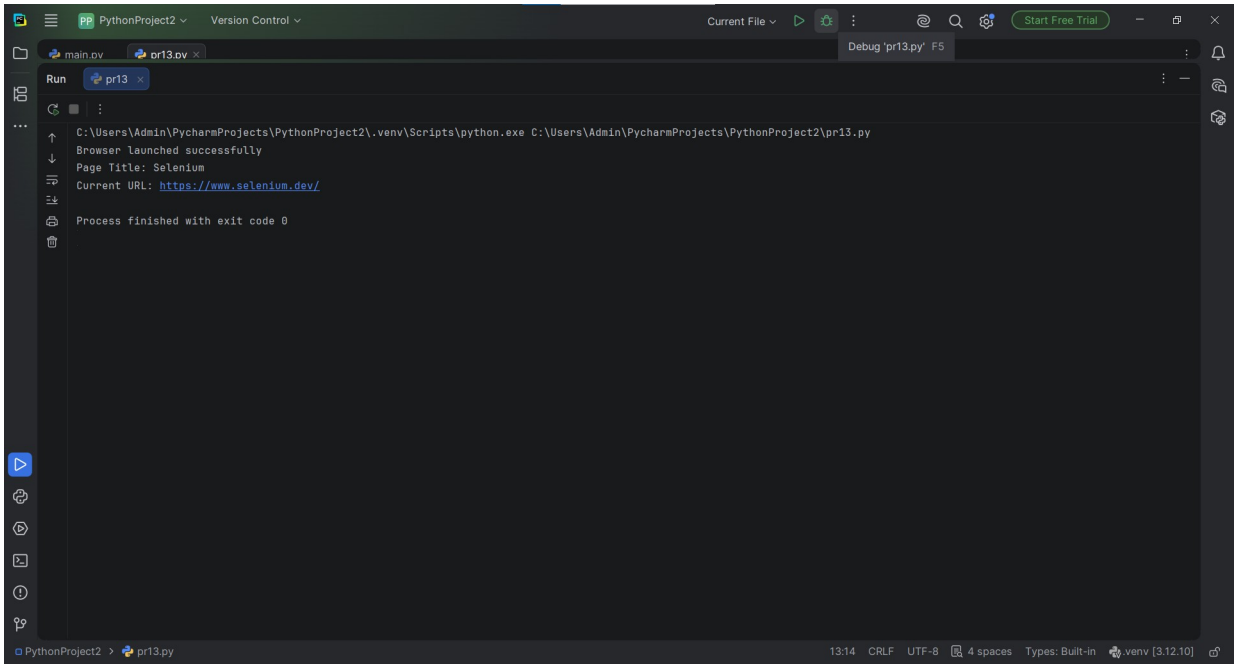
from selenium import webdriver	Imports Selenium's WebDriver interface.
webdriver.Chrome()	Starts a Chrome browser session.
driver.maximize_window()	Maximizes the browser window.
driver.get("https://www.selenium.dev/")	Opens the Selenium website.
print(...)	Displays a confirmation message, page title, and current URL.
driver.quit()	Closes the browser session.

7. Evidence Screenshots

The following screenshots were supplied as practical evidence. They show the Python script, Chrome opening the target site, the loaded Selenium homepage, and the PyCharm Run console.







8. Test Case Summary

Test ID	Test activity	Expected result	Observed result / status
TC-13-01	Launch Chrome	Chrome opens without WebDriver error.	Chrome launched; Pass
TC-13-02	Open Selenium website	Selenium homepage is displayed.	selenium.dev loaded; Pass
TC-13-03	Print page details	Title and URL appear in console.	Title: Selenium and URL printed; Pass
TC-13-04	Close browser	Browser session closes cleanly.	Process finished with exit code 0; Pass

9. Observations

- The Python script uses Selenium WebDriver to start and control Google Chrome.
- The browser opens the Selenium website at https://www.selenium.dev/.
- The Run console shows “Browser launched successfully”, the page title “Selenium”, and the current URL.
- The process finishes with exit code 0, indicating successful script completion.

10. Result

The Selenium environment test completed successfully. Chrome was launched, the Selenium website was opened, the page title and URL were printed in the console, and the browser session was closed.

11. Conclusion

This practical demonstrated the basic configuration and use of Selenium WebDriver with Python in PyCharm. The successful browser launch and console output confirm that the automation environment is functioning.